Guía de Implementación

JWT + Roles de Usuario

Node.js · Express · MongoDB · React

📌 **¿Qué lograrás?** Autenticación segura con JWT. Usuarios con rol "user" van a /productos y con rol "admin" al /panel-administrativo. Rutas protegidas tanto en frontend como en backend.

📋 **Orden de la guía** SECCIÓN 1 — Backend: instalar JWT y actualizar el controlador de login | SECCIÓN 2 — Middlewares: proteger las rutas del backend | SECCIÓN 3 — Pruebas con Postman: verificar el backend antes del frontend | SECCIÓN 4 — Frontend React: Context API + rutas protegidas | SECCIÓN 5 — Verificación final

SECCIÓN 1 — Configuración del Backend

Paso 1 — Instalar jsonwebtoken

Abre la terminal en la carpeta raíz de tu backend y ejecuta:

```
npm install jsonwebtoken
```

✅ **Verificar:** Debe aparecer en package.json: "jsonwebtoken": "^9.x.x"

Paso 2 — Agregar JWT_SECRET al .env

En la raíz de tu backend abre el archivo .env y agrega esta línea:

```
JWT_SECRET=mi_clave_super_secreta_2024
```

⚠️ **Importante:** NUNCA subas el .env a GitHub. Asegúrate de tenerlo en tu .gitignore.

Si no tienes .gitignore, créalo con este contenido:

```
node_modules/
.env
```

Paso 3 — Actualizar el modelo de la base de datos

```
import mongoose from "mongoose";

const UsersShema=new mongoose.Schema({
    Nombre:{type:String,required:true},
    Apellido:{type:String,required:true},
    telefono:{type:String,required:true,minlength:10},
    Correo_Electronico:{type:String,required:true},
    passwords:{type:String,required:true},
    rol: {type: String, enum: ['user', 'admin'],
        default: 'user'

    },
   codigoRecuperacion: String,
   codigoExpiracion: Date


});
// forzar que guarde en Users
const Users = mongoose.model("Users",UsersShema,"Users")

export default Users;
```

Paso 4 — Actualizar el controlador login.js

Revisar el controlador y realizar los ajustes necesarios para el buen funcionamiento

```
import bcrypt from "bcrypt";
import jwt from "jsonwebtoken";
import Users from "../models/User.js";

export const LoginUsuario = async (req, res) => {
    try {
        const { Correo_Electronico, passwords } = req.body;

        // Validamos que los campos estén presentes
        if (!Correo_Electronico || !passwords) {
            return res.status(400).json({ message: "Correo y contraseña obligatorios" });
        }

        // Buscamos el usuario en la base de datos
        const usuario = await Users.findOne({ Correo_Electronico });
        if (!usuario) {
            return res.status(404).json({ message: "Usuario no encontrado" });
        }
```

```javascript
// Comparamos la contraseña encriptada
        const passwordsValida = await bcrypt.compare(passwords, usuario.passwords);
        if (!passwordsValida) {
            return res.status(401).json({ message: "Contraseña incorrecta" });
        }

        // Generamos el token JWT con el rol incluido
        const token = jwt.sign(
            {
                id: usuario._id,
                rol: usuario.rol,
            },
            process.env.JWT_SECRET,
            { expiresIn: "1h" }
        );

        // Respondemos con el token y los datos del usuario
        res.status(200).json({
            message: "Inicio de sesión correcto",
            token,
            usuario: {
                id: usuario._id,
                nombre: usuario.Nombre,
                apellido: usuario.Apellido,
                email: usuario.Correo_Electronico,
                telefono: usuario.telefono,
                rol: usuario.rol,
            },
        });

    } catch (error) {
        res.status(500).json({ message: "Error al iniciar sesión", error: error.message });
    }
};
```

Paso 5 — Actualizar el controlador Users.js

```javascript
import Users from "../models/User.js";
import bcrypt from "bcrypt";

export const registrarUsers = async (req, res) => {
    try {
        const { Nombre, Apellido, telefono, Correo_Electronico, passwords } = req.body;

        // Validar campos obligatorios
        if (!Nombre || !Apellido || !telefono || !Correo_Electronico || !passwords) {
            return res.status(400).json({ message: "Todos los campos son obligatorios" });
        }

        // Verificar si ya existe
        const existeUsuario = await Users.findOne({ Correo_Electronico });
        if (existeUsuario) {
            return res.status(400).json({ message: "Usuario ya está registrado" });
        }

        // Encriptar contraseña
        const saltRounds = 10;
        const hashedPassword = await bcrypt.hash(passwords, saltRounds);

        // Crear usuario con rol fijo
        const nuevoUsuario = new Users({
            Nombre,
            Apellido,
            telefono,
            Correo_Electronico,
            passwords: hashedPassword,
            rol: "user"
        });

        await nuevoUsuario.save();

        res.status(201).json({ message: "Usuario registrado con éxito" });

    } catch (error) {
        res.status(500).json({
            message: "Error al registrar usuario",
            error: error.message
        });
    }
};

```

SECCIÓN 2 — Middlewares de Protección (Backend)

Los middlewares protegen las rutas del **backend**. Sin ellos, cualquier persona podría hacer peticiones directas a la API con Postman y obtener datos que no le corresponden, aunque el frontend los bloquee.

🔐 **Regla de oro:** Frontend protege la interfaz visual. Middlewares protegen los datos reales del backend. Ambos son necesarios.

Paso 4 — Crear el archivo auth.middleware.js

Crea la carpeta middlewares/ en la raíz de tu backend y dentro el archivo auth.middleware.js:

```javascript
// middlewares/auth.middleware.js
import jwt from "jsonwebtoken";
import Users from "../models/User.js";

// Verifica el token y consulta el usuario actualizado en BD
export const verificarToken = async (req, res, next) => {
    try {
        const authHeader = req.headers["authorization"];

        if (!authHeader || !authHeader.startsWith("Bearer ")) {
            return res.status(401).json({ message: "Token requerido" });
        }

        const token = authHeader.split(" ")[1];

        // Decodifica el token
        const decoded = jwt.verify(token, process.env.JWT_SECRET);

        // Consulta el usuario actualizado en la BD (por si cambió su rol o fue eliminado)
        const usuario = await Users.findById(decoded.id).select("-passwords");
        if (!usuario) {
            return res.status(401).json({ message: "Usuario no encontrado" });
        }

        // Guardamos el usuario completo en req para usarlo en los controladores
        req.usuario = usuario;
        next();

    } catch (error) {
        if (error.name === "TokenExpiredError") {
            return res.status(401).json({ message: "Token expirado, inicia sesión nuevamente" });
        }
        if (error.name === "JsonWebTokenError") {
            return res.status(401).json({ message: "Token inválido" });
        }
        res.status(500).json({ message: "Error en la autenticación", error: error.message });
    }
};

// Solo administradores
export const soloAdmin = (req, res, next) => {
    if (req.usuario?.rol !== "admin") {
        return res.status(403).json({ message: "Acceso denegado: se requiere rol admin" });
    }
    next();
};

// Solo usuarios
export const soloUser = (req, res, next) => {
    if (req.usuario?.rol !== "user") {
        return res.status(403).json({ message: "Acceso denegado: se requiere rol user" });
    }
    next();
};
```

Paso 5 — Aplicar middlewares en las rutas

El orden siempre debe ser: verificarToken primero → luego el de rol → luego el controlador.

Rutas de productos (solo rol user):

```javascript
import express from "express";
import { verificarToken,  soloAdmin } from "../middlewares/authMiddleware.js";
import { obtenerProductos, crearProducto, actualizarProducto,  eliminarProducto
} from "../controllers/Productos.js";

const router = express.Router();

// 👤 Ver productos (user y admin)
router.get("/", verificarToken, obtenerProductos);

// 👑 Crear
router.post("/", verificarToken, soloAdmin, crearProducto);

// 👑 Actualizar
router.put("/:id", verificarToken, soloAdmin, actualizarProducto);

// 👑 Eliminar
router.delete("/:id", verificarToken, soloAdmin, eliminarProducto);

export default router;
```

Rutas del panel admin (solo rol admin):

```js
import express from "express";
import { verificarToken,  soloAdmin } from "../middlewares/authMiddleware.js";
import { obtenerProductos, crearProducto, actualizarProducto,  eliminarProducto
} from "../controllers/Productos.js";

const router = express.Router();

// 👤 Ver productos (user y admin)
router.get("/", verificarToken, obtenerProductos);

// 🔥 Crear
router.post("/", verificarToken, soloAdmin, crearProducto);

// 🔥 Actualizar
router.put("/:id", verificarToken, soloAdmin, actualizarProducto);

// 🔥 Eliminar
router.delete("/:id", verificarToken, soloAdmin, eliminarProducto);

export default router;
```

Paso 5 — Validar el app server

Debe validar el app con las rutas nueva creadas

```js
import adminRoutes from './routes/admin.js';  // ← NUEVO
```

```js
app.use("/api/admin", adminRoutes);
```

Levantamos el servidor y todo tiene que quedar ok

SECCIÓN 3 — Pruebas con Postman

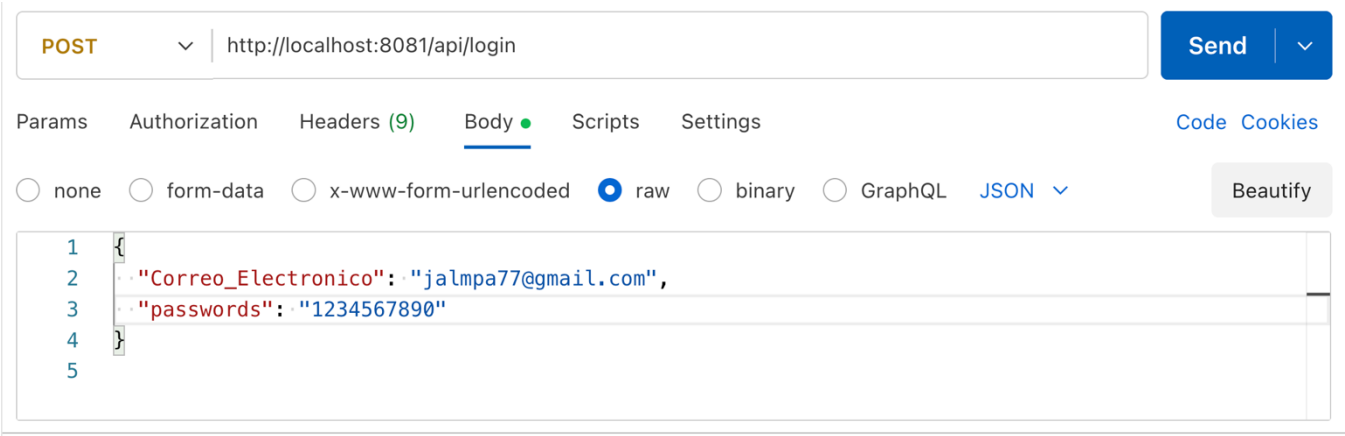
💡 **¿Por qué probar con Postman primero?** Postman te permite verificar que el backend funciona correctamente antes de conectar el frontend. Si algo falla aquí, es un problema del backend. Si aquí funciona pero falla en el navegador, es un problema del frontend.

Prueba 1 — Login y obtener el token

Esta es la prueba más importante. Primero debes obtener el token para usarlo en las demás pruebas.

Método	`POST`
URL	http://localhost:8081/api/login
Headers	`Content-Type: application/json`
Body	`raw → JSON`

Contenido del Body (pestaña Body → raw → JSON):



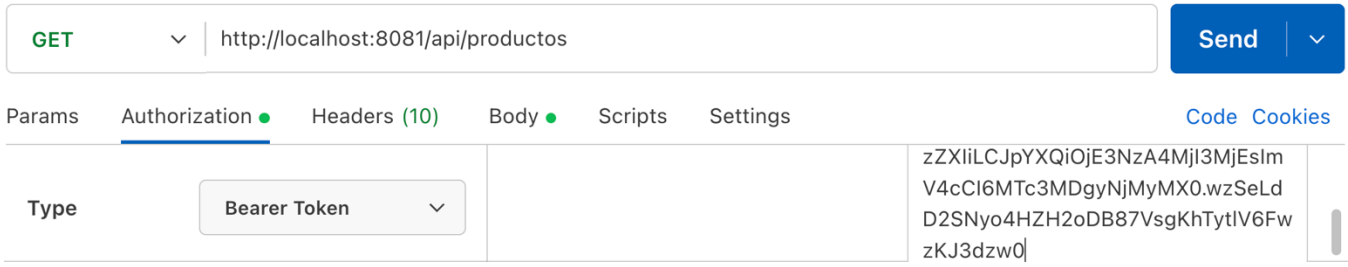
Respuesta esperada (200 OK):

📋 **Importante:** Copia el valor del campo "token" de la respuesta. Lo necesitarás en todas las pruebas siguientes.

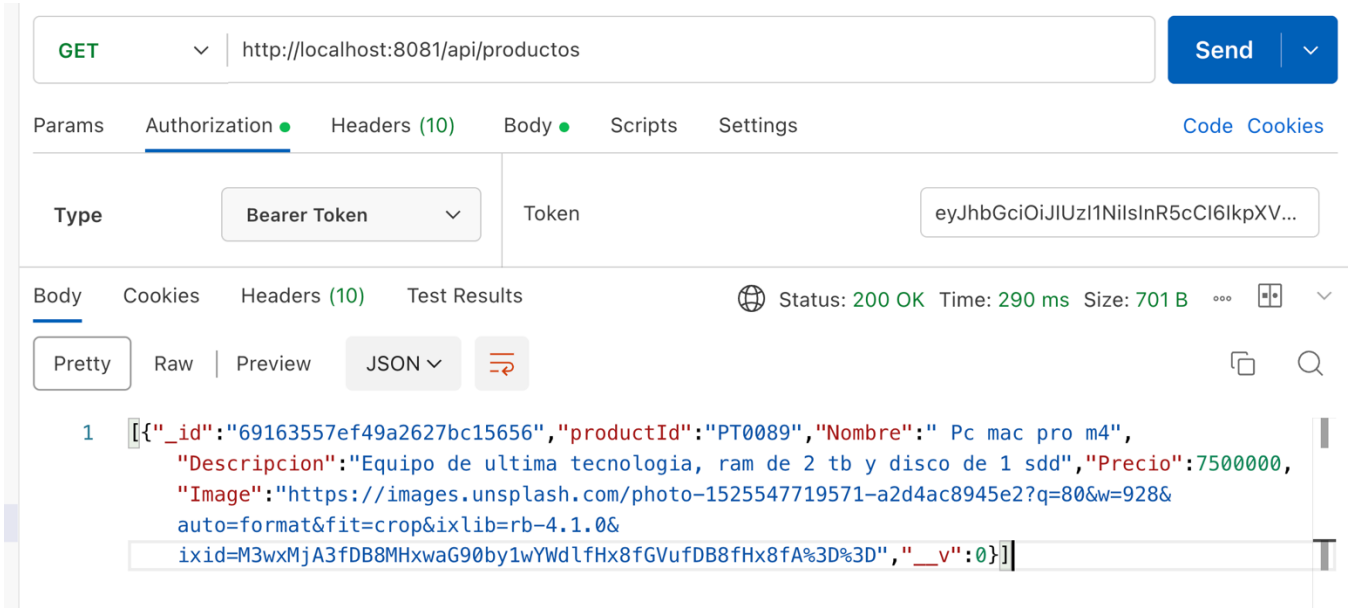
Prueba 2 — Acceder a una ruta protegida CON token

Así se envía el token en Postman para acceder a rutas que requieren autenticación:

Método	`GET`
URL	`http://localhost:8081/api/productos`
Pestaña	`Authorization`
Auth Type	`Bearer Token`
Token	`pega aquí el token que copiaste en la Prueba 1`



Le damos send y debe traer



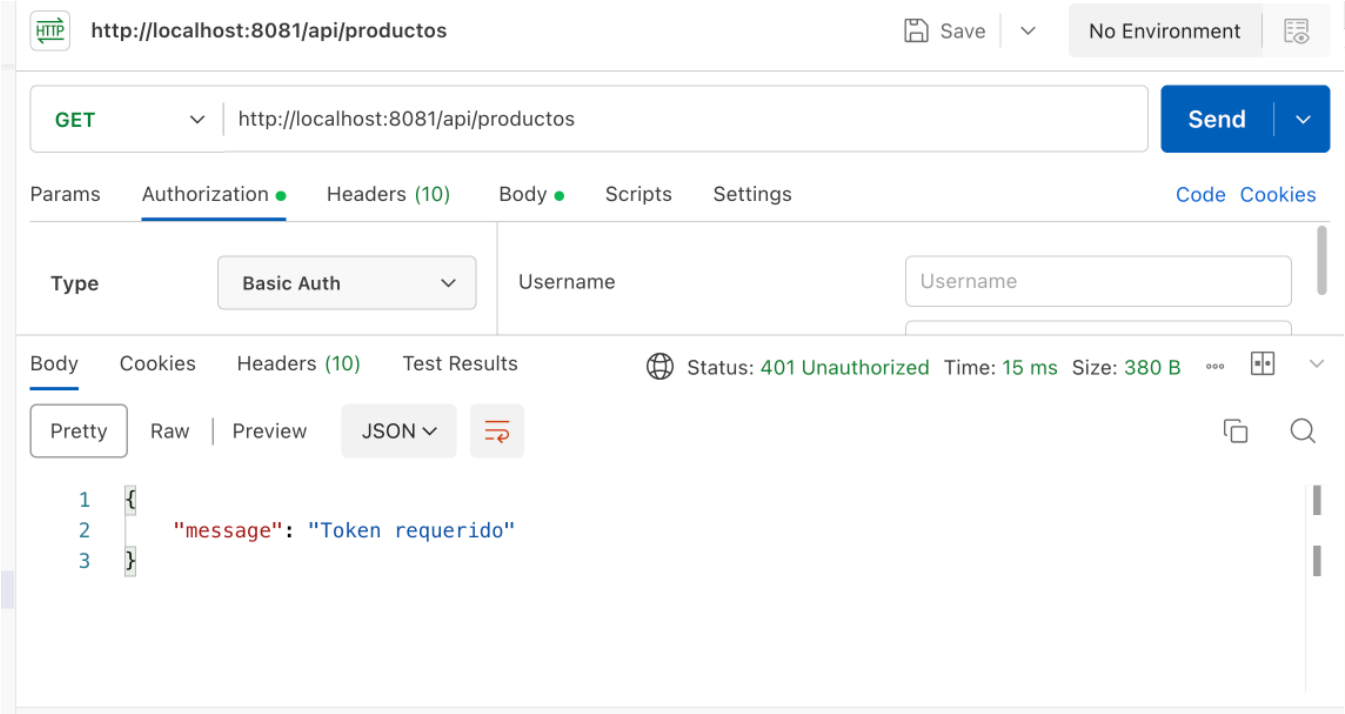
✅ **Resultado esperado:** Status 200 OK con los datos de los productos.

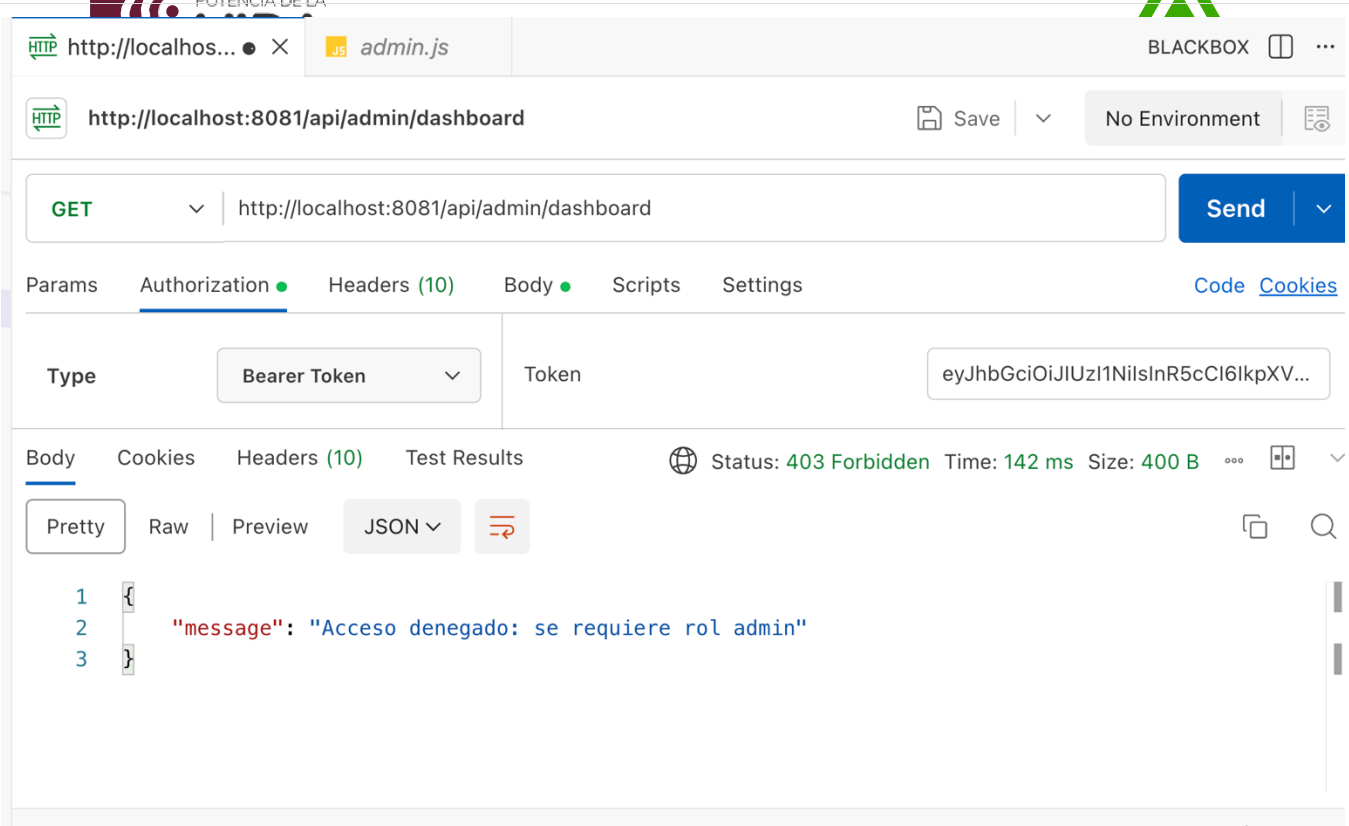
Prueba 3 — Verificar que los middlewares bloquean correctamente

Estas pruebas confirman que la seguridad funciona. Todas deben fallar con el error indicado:

Código	Situación	Mensaje esperado
401	Petición sin token	`Token requerido`
401	Token modificado o falso	`Token inválido`
401	Token expirado (pasadas 8h)	`Token expirado, inicia sesión nuevamente`
403	User accede a ruta de admin	`Acceso denegado: se requiere rol admin`
403	Admin accede a ruta de user	`Acceso denegado: se requiere rol user`
200	Token válido + rol correcto	`Datos de la ruta correctamente`

💡 **Tip Postman — cómo probar sin token:** En la pestaña Authorization selecciona "No Auth" y envía la petición. Debe responder 401 con el mensaje "Token requerido".

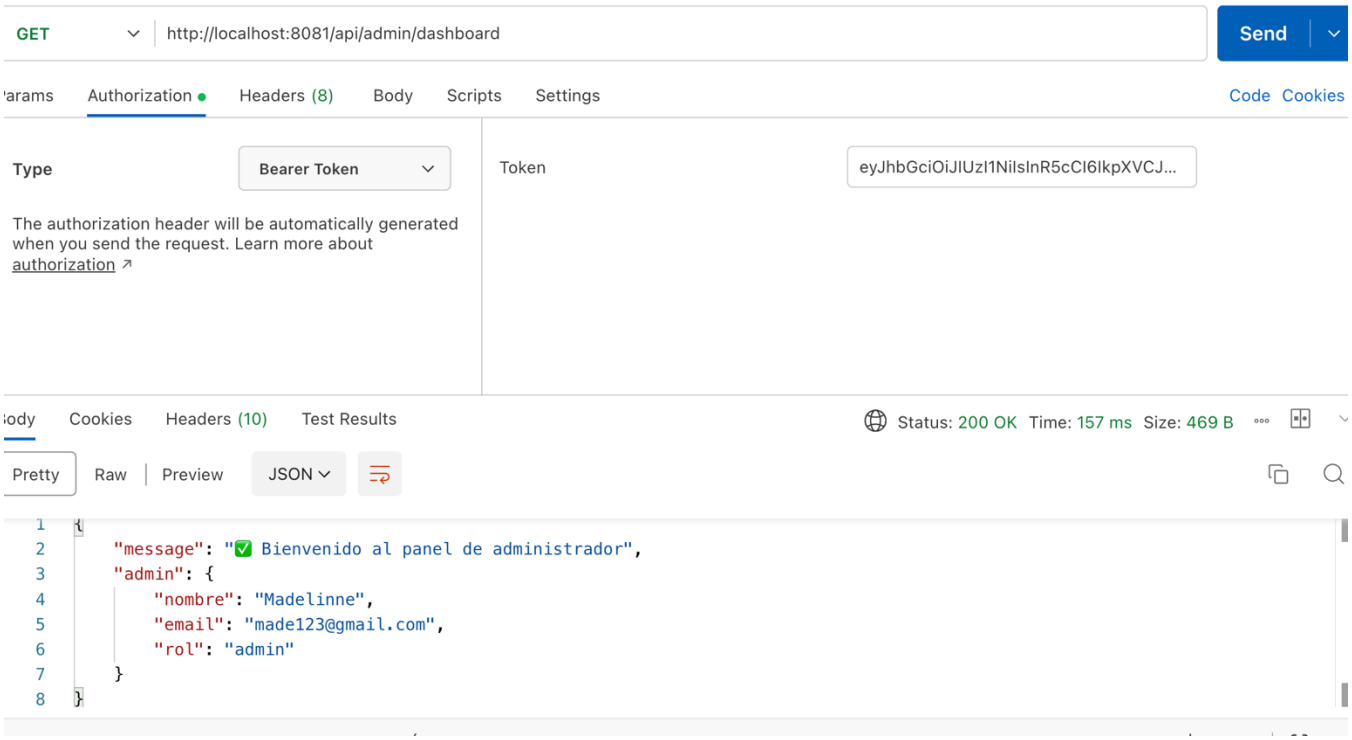




Ingreso al panel administrativo

Van a crear un usuario normal por Portman ingresa a la base. De datos y cambiar el rol por admin
Y listo ya tiene usuario con admin

2. Conectando la ruta admin



✅ **Resultado:** Cada vez que hagas login, el token se actualiza automáticamente en todas tus peticiones.

SECCIÓN 4 — Configuración del Frontend (React)

✅ **Antes de continuar:** Asegúrate de que las Pruebas 1, 2 y 3 de Postman funcionan correctamente antes de seguir con el frontend.

SECCIÓN 1 — Configurar la carpeta context desde de src

```jsx
import { BrowserRouter, Routes, Route, Navigate } from 'react-router-dom';
import { AuthProvider } from "./context/AuthContext.jsx";
import PrivateRoute from './components/PrivateRoute.jsx';
import Login from './components/Auth/Login';           // ← Login
import Register from './components/Auth/Register';
import AdminPanel from './components/Auth/Admin';      // ← AdminPanel (no "Login")
import Home from './components/Pages/Home';
import Layout from './components/Layout/Layout';

function App() {
  return (
    <BrowserRouter>
      <AuthProvider>
        <Routes>
          <Route path="/" element={<Home />} />
          <Route path="/login" element={<Login />} />
          <Route path="/register" element={<Register />} />

          <Route
            path="/productos"
            element={
              <PrivateRoute rolRequerido="user">
                <Layout />
                <div>Página de Productos</div>
              </PrivateRoute>
            }
          />

          <Route
            path="/admin"
            element={
              <PrivateRoute rolRequerido="admin">
                <AdminPanel />  {/* ← Usa AdminPanel */}
              </PrivateRoute>
            }
          />

          <Route path="*" element={<Navigate to="/" replace />} />
        </Routes>
      </AuthProvider>
    </BrowserRouter>
  );
}

export default App;
```

Crear un archivo llamado

PrivateRoute.jsx

```jsx
// src/context/AuthContext.jsx
import { createContext, useContext, useState } from "react";
import { useNavigate } from "react-router-dom";
import axios from "axios";  // ← IMPORTAR AXIOS

const AuthContext = createContext();

export const AuthProvider = ({ children }) => {
    const [usuario, setUsuario] = useState(null);
    const navigate = useNavigate();

    const login = async (correo, password) => {
        try {
            const response = await axios.post("http://localhost:8081/api/login", {
                Correo_Electronico: correo,
                passwords: password,
            });

            const data = response.data;  // axios ya parsea el JSON automáticamente

            // Guardamos el usuario y token en memoria
            setUsuario({
                ...data.usuario,
                token: data.token,
            });

            // Redirigimos según el rol
            if (data.usuario.rol === "admin") {
                navigate("/admin");
            } else {
                navigate("/productos");
            }

        } catch (error) {
            // Manejo de errores con axios
            if (error.response) {
                throw new Error(error.response.data.message || 'Error al iniciar sesión');
            } else if (error.request) {
                throw new Error('No se pudo conectar con el servidor');
            } else {
                throw new Error('Error al procesar la solicitud');
            }
        }
    };

    const logout = () => {
        setUsuario(null);
        navigate("/login");
    };

    return (
        <AuthContext.Provider value={{ usuario, login, logout }}>
            {children}
        </AuthContext.Provider>
    );
};

export const useAuth = () => {
    const context = useContext(AuthContext);
    if (!context) {
        throw new Error("useAuth debe usarse dentro de un AuthProvider");
    }
    return context;
};
```

En React, el Context se usa para compartir información global entre componentes sin tener que pasar props manualmente por cada nivel.

¿Para qué se usa en un login?

Normalmente se usa para manejar:

- Usuario autenticado
- Token
- Rol (admin / user)
- Función de login y logout

El PrivateRoute sirve para **proteger rutas**.

Es decir:

Si NO está logueado → lo manda al login
Si está logueado → lo deja entrar

SECCIÓN 2 — Configurar y validar el componente Login

```
import { useState } from "react";
import { useNavigate } from 'react-router-dom';
import { useAuth } from '../../context/AuthContext';  // ← IMPORTAR
import { User, Mail, Lock, Eye, EyeOff, LogIn, Loader2, Shield } from "lucide-react";

export default function Login() {

    const [email, setEmail] = useState('');
    const [password, setPassword] = useState('');
    const [showPassword, setShowPassword] = useState(false);
    const [loading, setLoading] = useState(false);
    const [message, setMessage] = useState({ type: '', text: '' });
    const navigate = useNavigate();
    const { login } = useAuth();  // ← USAR EL CONTEXTO

    async function handleSubmit(e) {
        e.preventDefault();
        setLoading(true);
        setMessage({ type: '', text: '' });

```

```
try {
            // ← USAR login() del contexto que redirige según el rol
            await login(email, password);

            setMessage({
                type: 'success',
                text: '¡Inicio de sesión correcto!'
            });

        } catch (error) {
            console.error('Error:', error);
            setMessage({
                type: 'error',
                text: error.message || 'Error al iniciar sesión'
            });
        } finally {
            setLoading(false);
        }
    }
```

```
return (
        <main className="min-h-screen flex items-center justify-center px-4 py-12 bg-linear-to-br from-blue-50 via-white to-purple-50">
            <div className="w-full max-w-md">

                <div className="bg-white rounded-2xl shadow-2xl p-8 border border-gray-100">

                    {/* Título */}
                    <div className="text-center mb-8">
                        <div className="inline-flex items-center justify-center w-16 h-16 bg-linear-to-r from-blue-600 to-purple-600 rounded-full mb-4">
                            <User className="w-8 h-8 text-white" />
                        </div>
                        <h2 className="text-3xl font-bold text-gray-900 mb-2">¡Bienvenido de vuelta!</h2>
                        <p className="text-gray-600">Inicia sesión en tu cuenta</p>
                    </div>

                    <form onSubmit={handleSubmit} className="space-y-6">

                        {/* Email */}
                        <div>
                            <label htmlFor="email" className="block text-sm font-medium text-gray-700 mb-2">
                                <Mail className="w-4 h-4 inline mr-2 text-gray-400" />
                                Correo electrónico
                            </label>
                            <input
                                type="email"
                                id="email"
                                placeholder="tu@email.com"
                                onChange={e => setEmail(e.target.value)}
                                value={email}
                                className="w-full px-4 py-3 border border-gray-300 rounded-lg focus:ring-2 focus:ring-blue-500 focus:border-blue-500 transition-all duration-200 text-gray-900"
                                required
                            />
                        </div>
```

SECCIÓN 3 — Crear el componente admin